

CS434 – NEURAL NETWORK & DEEP LEARNING

Cao Tien Dung, PhD

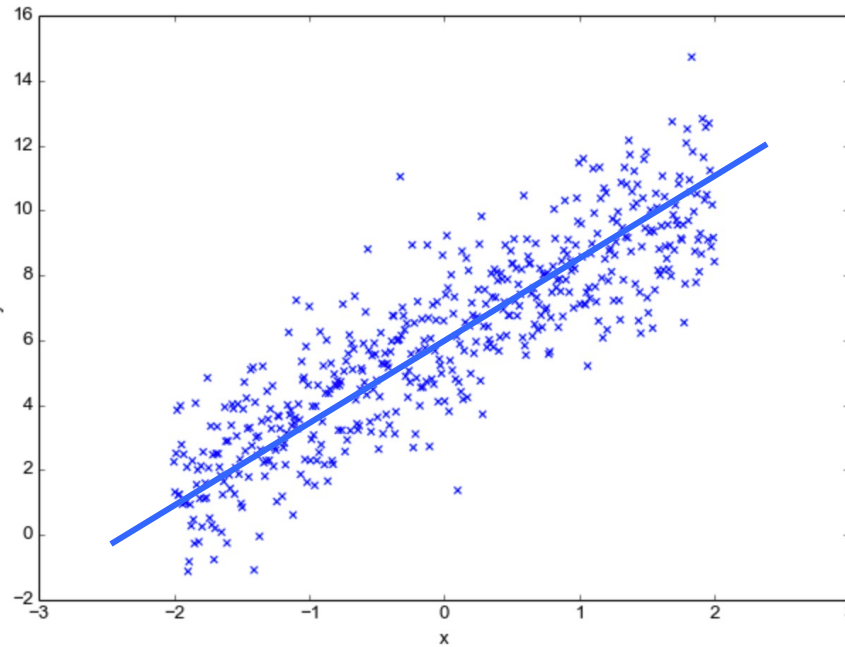




OUTLINE

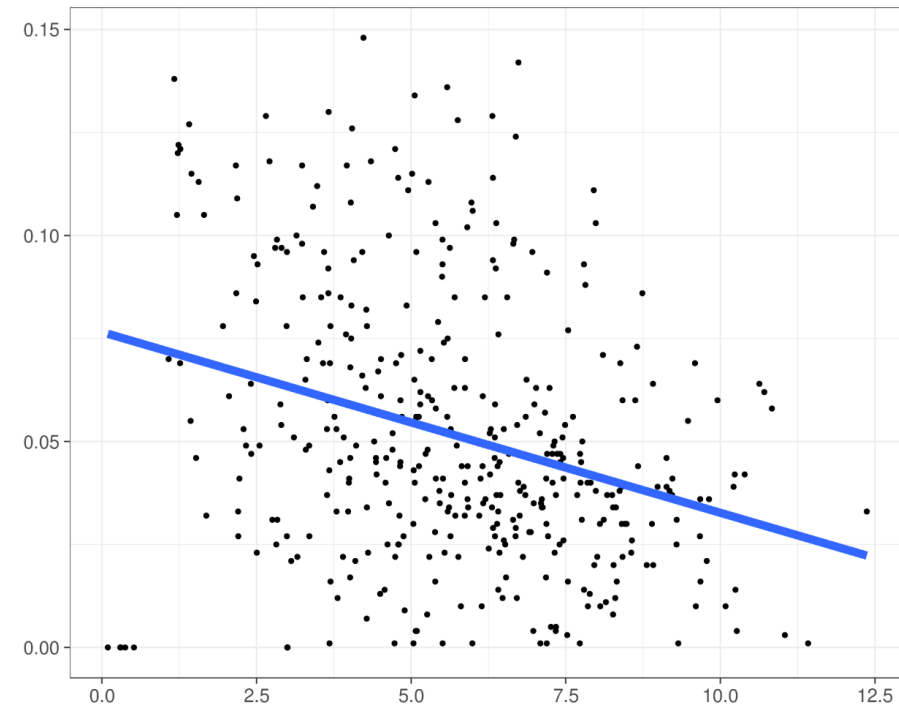
- Neural Network
- Tensorflow
- Lab

LINEAR (LOGISTICS) REGRESSION



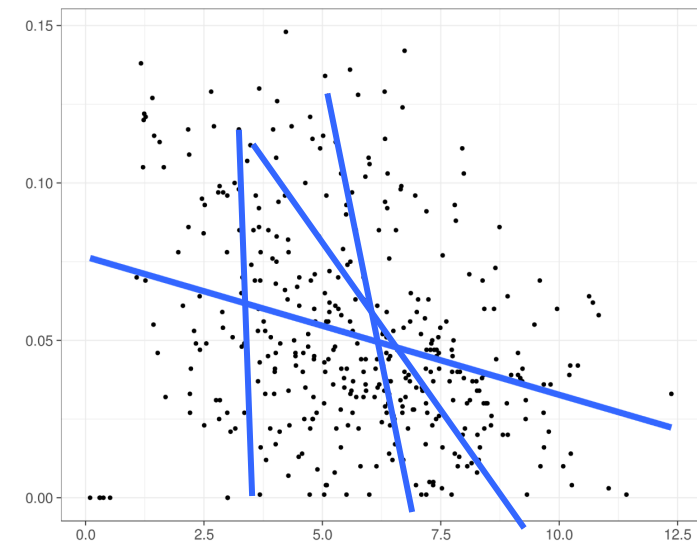
→ acceptable???

1 function cannot cover
feature of data???



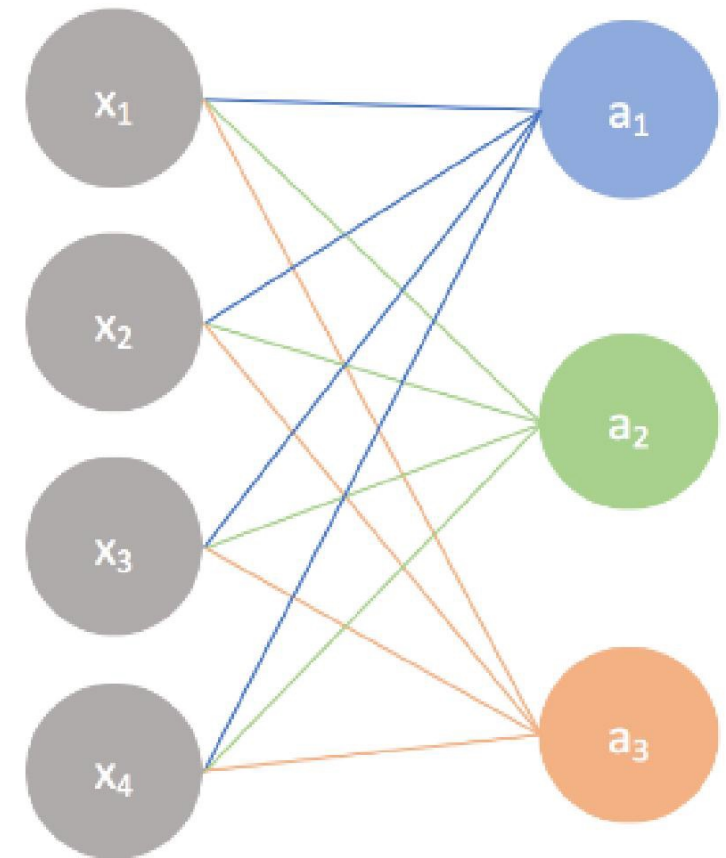
NEURAL FUNCTION

- Brain function occurs as the result of the firing of neurons.
- Different areas of the brain have different functions.
- Multiple brain functions can cover the diversity of this data.
 - Function is activated if it covers data point features

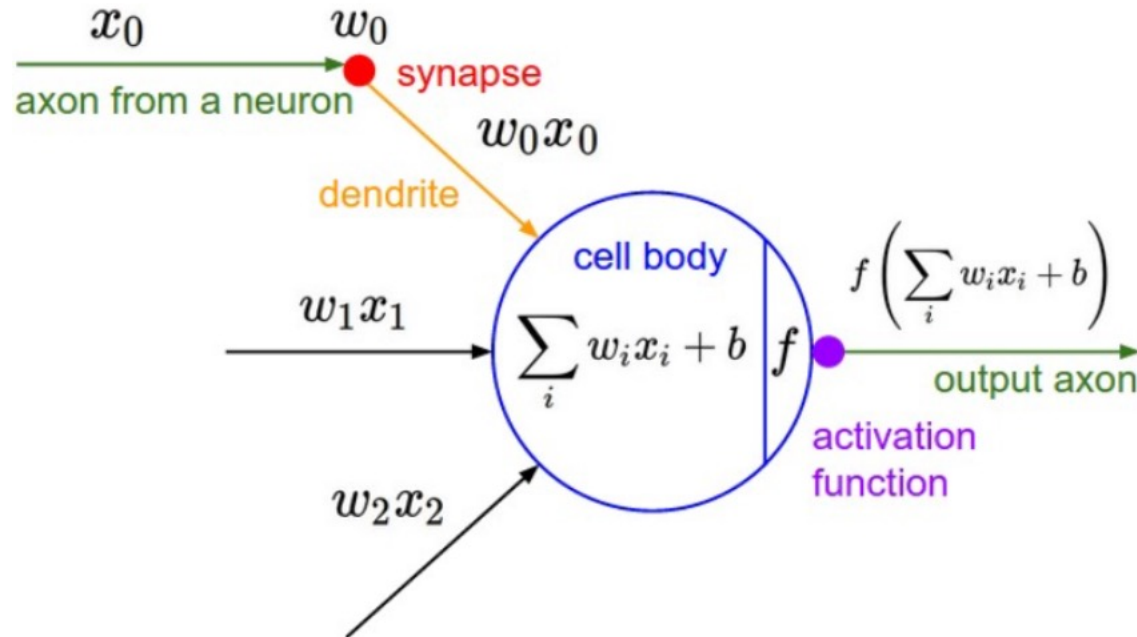


NEURAL NETWORKS

- Neural networks are made up of nodes (i.e., neuron), connected by links
- Each link has associated weight and activation level.
- Each node represent a function (typically combining a linear function -summing over weighted inputs- with an activation function) to produce output.



MATH MODEL OF NEURON



$X = (x_0, x_1, x_2) \rightarrow$ input data vector

$W = (w_0, w_1, w_2) \rightarrow$ weight vector

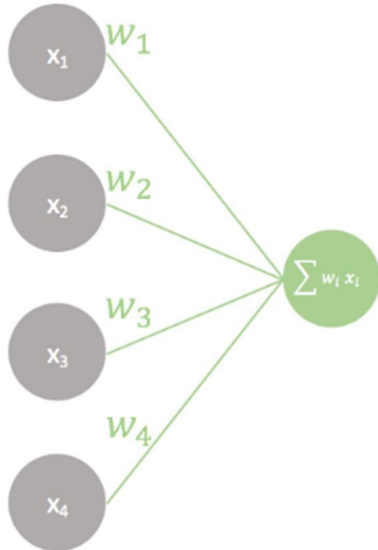
$b \rightarrow$ bias

$y = f(W X^T + b) \rightarrow$ output

$f \rightarrow$ activation function

ACTIVATION FUNCTION

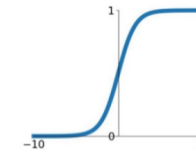
- Some functions like: ReLU, Sigmoid, Softmax, Tanh, ...
 - Capture non-linear relationship between input and output
 - Map linear function to useful output area



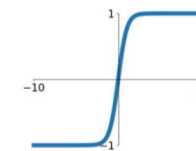
$$f_w(x) = \begin{cases} \sum w_i x_i \geq \theta \rightarrow \text{neuron fires} \\ \sum w_i x_i < \theta \rightarrow \text{neuron doesn't fire} \end{cases}$$

Sigmoid

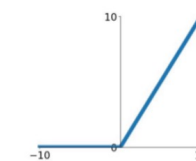
$$\sigma(x) = \frac{1}{1+e^{-x}}$$



tanh
 $\tanh(x)$

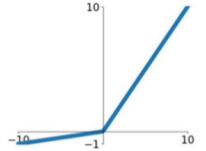


ReLU
 $\max(0, x)$



Leaky ReLU

$$\max(0.1x, x)$$

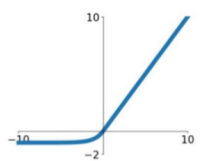


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

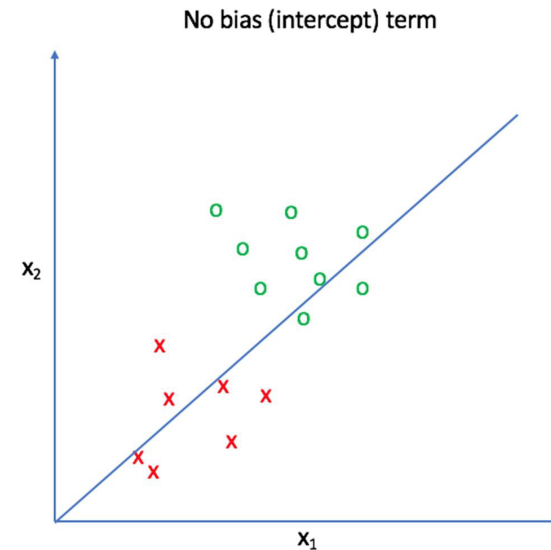
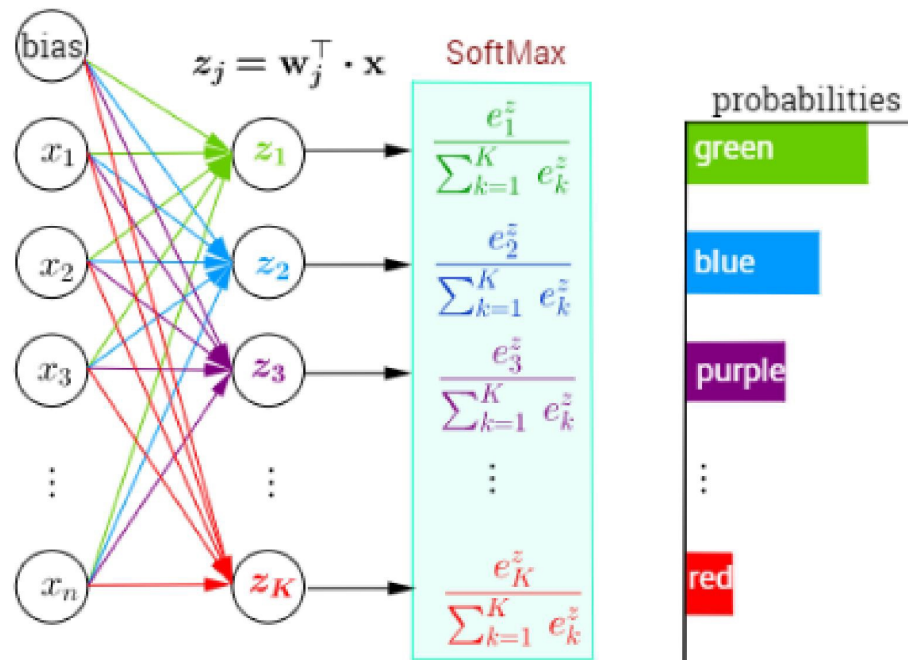
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

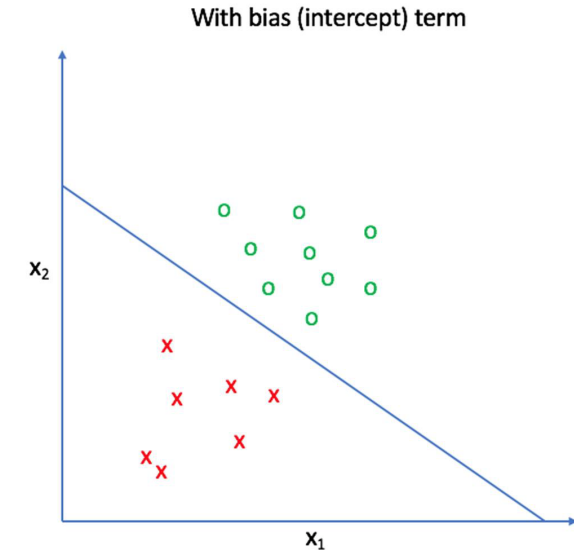


BIAS

- Bias make better fit



Our line is forced to pass through the origin.

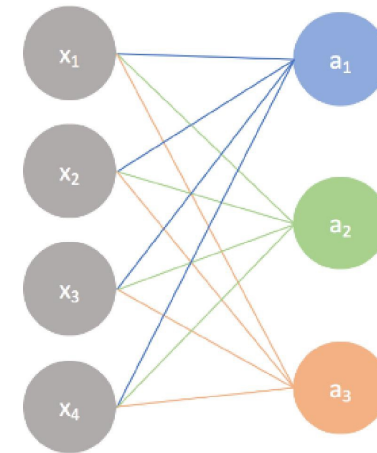


Adding the intercept term allows for much better fit.

SIMPLE NEURAL NETWORK

- One sample of data observation

$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} = \begin{bmatrix} w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \\ w_1x_1 + w_2x_2 + w_3x_3 + w_4x_4 + b \end{bmatrix} \xrightarrow{\text{activation}} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}$$

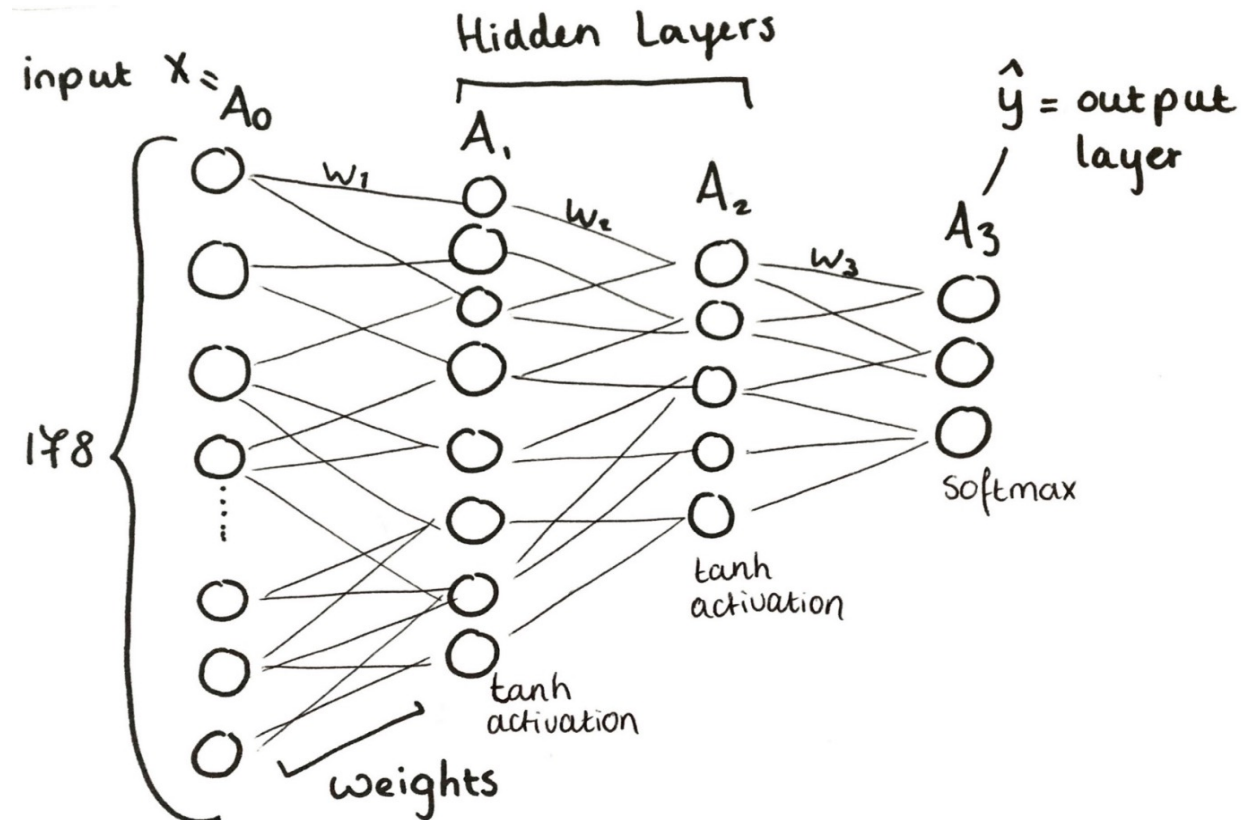


- Multiple samples of data observation

$$\begin{bmatrix} w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \\ w_1 & w_2 & w_3 & w_4 \end{bmatrix} \begin{bmatrix} \text{Observation 1} & \text{Observation 2} & \text{Observation 3} & \text{Observation 4} \\ x_1 & x_1 & x_1 & x_1 \\ x_2 & x_2 & x_2 & x_2 \\ x_3 & x_3 & x_3 & x_3 \\ x_4 & x_4 & x_4 & x_4 \end{bmatrix} + \begin{bmatrix} b \\ b \\ b \end{bmatrix} \xrightarrow{\text{activation}} \begin{bmatrix} \text{Observation 1} & \text{Observation 2} & \text{Observation 3} & \text{Observation 4} \\ a_1 & a_1 & a_1 & a_1 \\ a_2 & a_2 & a_2 & a_2 \\ a_3 & a_3 & a_3 & a_3 \end{bmatrix}$$

DEEP NEURAL NETWORK

- Adding multiple hidden layers between input and output

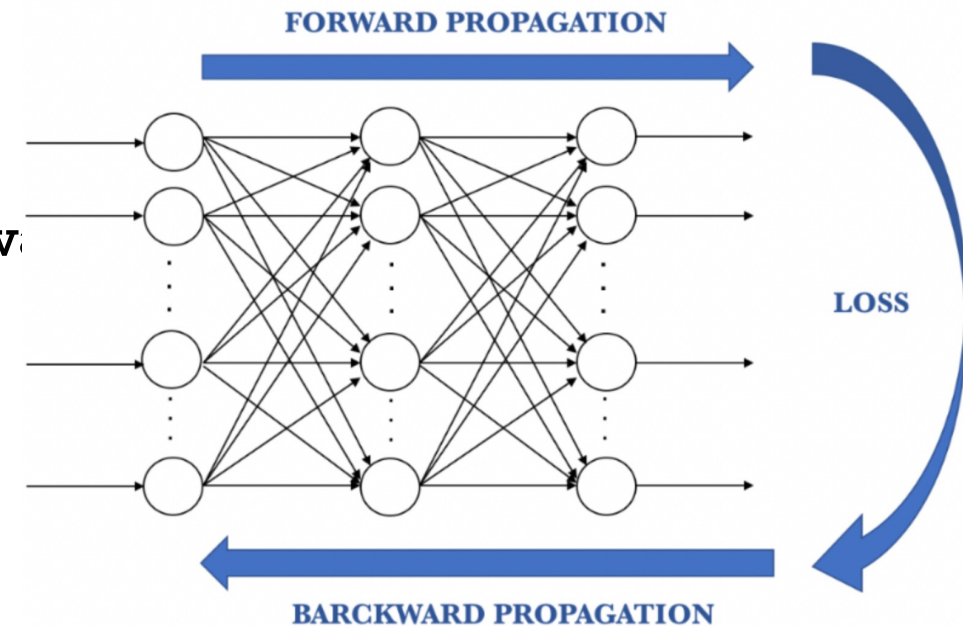


DEEP NEURAL NETWORK

- $Y = f_{l_n}(W_n(f_{l_{n-1}}(W_{n-1}(\dots f_{l_2}(W_2(f_{l_1}(W_1X^T + B_1)^T + B_2)\dots)^T + B_{n-1})^T + B_n)$
 - $f_{l_{1\dots n}}$ are the activation functions
- Each layer may have different activation function as well as bias
- Size of W_i and B_i depend on number of neuron for each layer.
- Define how many layer and number of neuron for each layer called defining architecture of DNN

DNN LEARNING

1. Start with random network parameters
2. Pass input through network
3. Compare predicted value with expected value



4. Perform the back propagation to update net parameters with gradient descent
5. Continue iterating from step 2 to 4 until we have a good model.

LOSS FUNCTION

- Choosing loss function usually depends on the activation function that you choose in the network. But 2 typical loss functions are used:
 - Mean Square Error: $\frac{1}{2} * \sum (y - y_p)^2$ for regression problem
 - Cross-Entropy: measuring error between two probability distributions is used for classification problem (both sigmoid and softmax)

$loss = - \sum_i^C t_i \log(f(s)_i)$, C is number of classes, t_i expected value represented by one-hot vector

- Contrastive Loss $\ell_{i,j} = -\log \frac{\exp(\text{sim}(z_i, z_j)/\tau)}{\sum_{k=1}^{2N} [k \neq i] \exp(\text{sim}(z_i, z_k)/\tau)}$,

UPDATE PARAMETER

- Network parameters are updated by gradient descent like this

$$W_{i,t+1} = W_{i,t} - \alpha \frac{\partial L}{\partial W_{i,t}}$$

- i is layer i^{th}
- t is iteration t^{th}
- α is learning rate

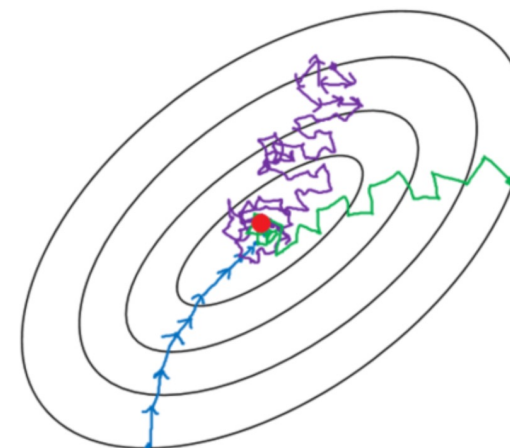
- Derivative at layer i^{th}

$$\frac{\partial L}{\partial W_i} = \frac{\partial L}{\partial g_i} \frac{\partial g_i}{\partial f_i} \frac{\partial f_i}{\partial W_i}$$

- g_i is activation function
- $f_i = W_i X^T$

TRAINING NN

- We usually use gradient descent to update weights of NN.
- A training dataset can be divided into one or more batches.
 - **Batch Gradient Descent.** Batch Size = Size of Training Set
 - **Stochastic Gradient Descent.** Batch Size = 1
 - **Mini-Batch Gradient Descent.** $1 < \text{Batch Size} < \text{Size of Training Set}$



- Batch gradient descent
- Mini-batch gradient Descent
- Stochastic gradient descent



BATCH GRADIENT DESCENT

- Updates the model after all training examples have been evaluated.
- Upsides:
 - Fewer updates
 - Reduce gradient error
- Downsides
 - Model updates are slow for large training dataset.
 - All dataset must be available on memory



STOCHASTIC GRADIENT DESCENT

- The variation of the gradient descent of algorithm and model updates are computed for each data sample.
- Upsides
 - Result in faster learning on some problems (online learning).
 - The noisy update may help the model to avoid local minima.
- Downsides
 - Take longer to train models on large datasets.
 - The noisy update make hard to settle an error minimum for lost function.

MINI-BATCH GRADIENT DESCENT

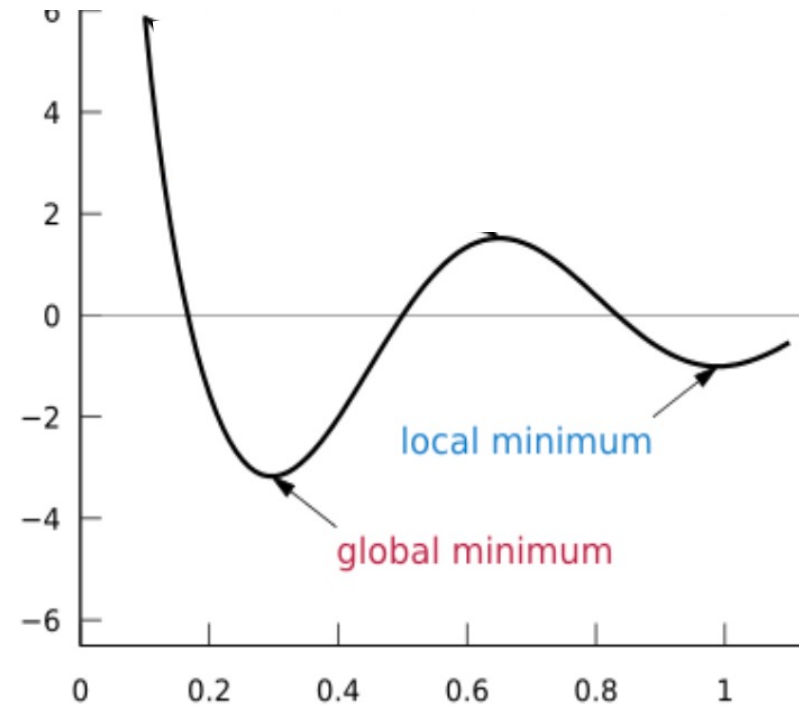
- The variation and model update are computed on a subset of training dataset.
- Upside
 - Avoiding local minima because of higher update frequency
 - More efficient process than stochastic gradient descent.
 - Both the efficiency of not having all training data in memory and algorithm implementations.
- Downside
 - Require configuration of “mini-batch size”

EPOCH?

- Number of loop proceeds over the training dataset.
- Batch vs Epoch
 - The batch size is a number of samples processed before the model is updated.
 - Size of batch must between 1 and size of training dataset
 - The number of epochs is the number of complete passes through the training dataset.
 - Number of epochs is any from 1 to infinity.

OPTIMIZATION ALGORITHMS

- Problem of local optima.
 - Choosing a proper learning rate can be difficult.





OPTIMIZATION ALGORITHMS

- Momentum
- Nesterov accelerated gradient
- Adagrad
- Adadelta
- RMSprop
- **Adam (most commonly used)**
- AdaMax
- See here: <http://runder.io/optimizing-gradient-descent/index.html>

TENSORFLOW



WHAT IS TENSORFLOW

- TensorFlow is a deep learning library recently open-sourced by Google.
- But what does it actually do?
 - TensorFlow provides primitives for **defining functions on tensors** and **automatically computing their derivatives**.



BUT WHAT IS TENSOR?

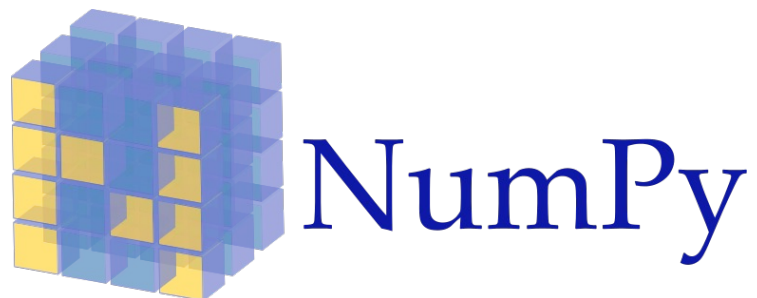
- Formally, tensors are multilinear maps from vector spaces to the real numbers (V vector space, and V^* dual space)

$$f : \underbrace{V^* \times \dots \times V^*}_{p \text{ copies}} \times \underbrace{V \times \dots \times V}_{q \text{ copies}} \rightarrow \mathbb{R}$$

- A scalar is a tensor ($f : \mathbb{R} \rightarrow \mathbb{R}$, $f(e_1) = c$)
- A vector is a tensor ($f : \mathbb{R}^n \rightarrow \mathbb{R}$, $f(e_i) = v_i$)
- A matrix is a tensor ($f : \mathbb{R}^n \times \mathbb{R}^m \rightarrow \mathbb{R}$, $f(e_i, e_j) = A_{ij}$)
- Common to have fixed basis, **so a tensor can be represented as a multidimensional array of numbers.**

TENSORFLOW VS NUMPY

- Few people make this comparison, but TensorFlow and Numpy are quite similar. (Both are N-d array libraries!)
- Numpy has N-d array support, but doesn't offer methods to create tensor functions and automatically compute derivatives (+ no GPU support).



TF2 VS NP

```
▶ In [1]: import numpy as np  
a = np.zeros((2,2))  
b = np.ones((2,2))  
np.sum(b, axis=1)
```

```
Out[1]: array([2., 2.])
```

```
In [2]: a.shape
```

```
Out[2]: (2, 2)
```

```
In [3]: np.reshape(a, (1,4))
```

```
Out[3]: array([[0., 0., 0., 0.]])
```

```
In [8]: import tensorflow as tf  
a = tf.zeros((2,2))  
b = tf.ones((2,2))  
tf.reduce_sum(b, 1)
```

```
Out[8]: <tf.Tensor: shape=(2,), dtype=float32, numpy=array([2., 2.], dtype=float32)>
```

```
In [9]: a.get_shape()
```

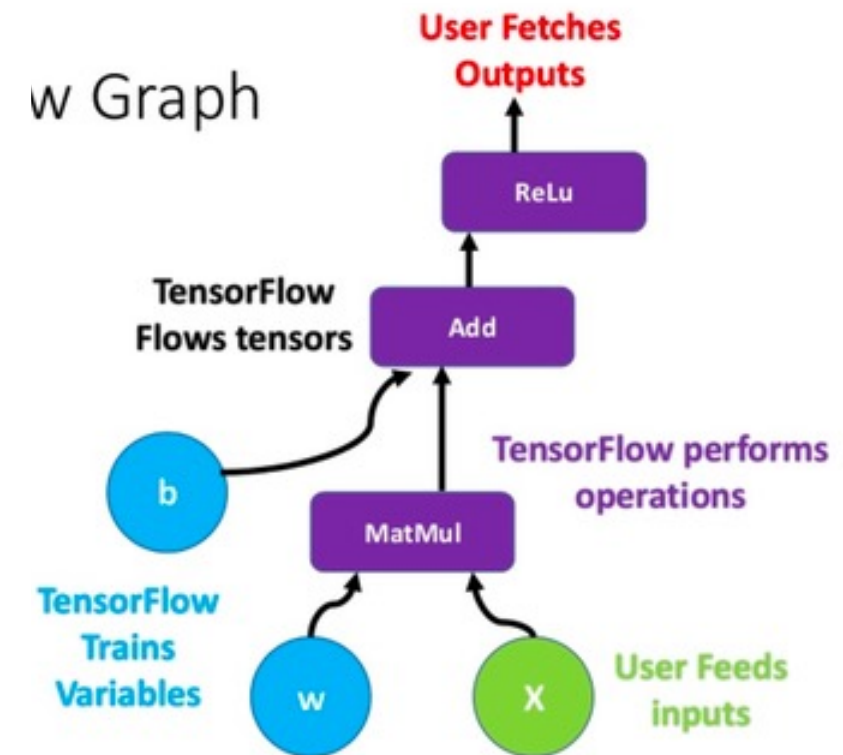
```
Out[9]: TensorShape([2, 2])
```

```
In [10]: tf.reshape(a, (1, 4))
```

```
Out[10]: <tf.Tensor: shape=(1, 4), dtype=float32, numpy=array([[0., 0., 0., 0.]], dtype=float32)>
```

TENSORFLOW COMPUTATION GRAPH

- “TensorFlow programs are usually structured into a construction phase, that assembles a graph, and an execution phase that uses a session to execute ops in the graph.” - TensorFlow docs
- All computations add nodes to global default graph (docs)



TF1: BUILD A GRAPH, THEN RUN IT.

```
import tensorflow as tf # 1.14.0
print(tf.__version__)

x = tf.constant(1)
y = tf.constant(2)
z = tf.add(x, y)

print(z) # Tensor("Add:0", shape=(), dtype=int32)

with tf.Session() as sess:
    print(sess.run(x)) # 3
```



Check TF2 vs TF1x: https://www.tensorflow.org/guide/effective_tf2

THREE MODEL BUILDING STYLES

- Sequential models (no difference between TF1x & TF2)

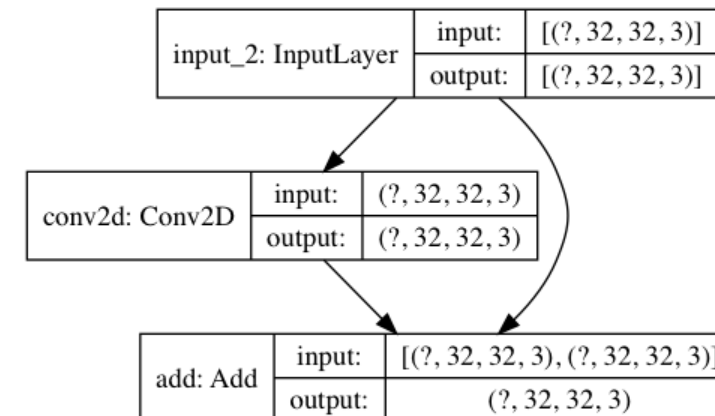
```
model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(512, activation='relu'),
    tf.keras.layers.Dropout(0.2),
    tf.keras.layers.Dense(10, activation='softmax')
])
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])

model.fit(x_train, y_train, epochs=5)
model.evaluate(x_test, y_test)
```

THREE MODEL BUILDING STYLES

- Functional models

```
from tensorflow import keras
from tensorflow.keras import layers
inputs = keras.Input(shape=(32, 32, 3))
y = layers.Conv2D(3, (3, 3), activation='relu', padding='same')(inputs)
outputs = layers.add([inputs, y])
model = keras.Model(inputs, outputs)
keras.utils.plot_model(model, 'skip_connection.png', show_shapes=True)
```



THREE MODEL BUILDING STYLES

- Subclassed models

```
# from tensorflow import keras
# from tensorflow.keras import layers
class MyModel(keras.Model):
    def __init__(self, num_classes=10):
        super(MyModel, self).__init__(name='my_model')
        self.dense_1 = layers.Dense(32, activation='relu')
        self.dense_2 = layers.Dense(num_classes, activation='sigmoid')

    def call(self, inputs):
        # Define your forward pass here
        x = self.dense_1(inputs)
        return self.dense_2(x)
```

TWO TRAINING STYLES

- Build-in

```
optimiser = tf.keras.optimizers.Adam()  
model.compile(optimizer=optimiser,  
              loss='sparse_categorical_crossentropy',  
              metrics=['accuracy'])  
  
model.fit(x_train, y_train, epochs=5)
```

- Or, define your own

```
model = MyModel()  
  
with tf.GradientTape() as tape:  
    logits = model(images)  
    loss_value = loss(logits, labels)  
  
grads = tape.gradient(loss_value, model.trainable_variables)  
optimizer.apply_gradients(zip(grads, model.trainable_variables))
```



LET TRY

- Fraud Detection Case Study Using Neural Network
 - At two level: low level and keras API.
 - <https://www.tensorflow.org/guide/core?authuser=1>